

Project Report

Web -Based Remote Monitoring, Electricity Cost Modeling, and Decision Support System Powered by Artificial Intelligence

By

Sk. Zayed Mahmood

Email: sk.zayedmahmood@gmail.com

Table of Contents

1. Project Overview	2
2. Dataset Overview	2
3. Data Cleaning and Preprocessing	4
3.1 Converting the Date Column	4
3.2 Setting the Date Index	5
3.3 Sorting the Dataset	5
3.4 Checking Chronological Order	5
3.5 Checking Time Intervals	6
3.6 Missing Value Analysis	7
3.7 Duplicate Value Investigation	7
3.8 Removal of Non-Predictive Columns	9
3.9 Outlier Detection and Treatment	10
4. Exploratory Data Analysis (EDA)	10
4.1 Electricity Demand Over Time	10
4.2 Distribution of Electricity Demand	11
4.3 Generation versus Electricity Demand	12
4.4 Correlation Analysis	14
5. Feature Engineering and Data Preparation	15
5.1 Time-Based Feature Extraction	15
5.2 Lag and Rolling Features	16
5.3 Removal of Missing Rows	17
5.4 Data Preparation for Modelling	17
5.5 Data Scaling for LSTM	18
6. Model Development	19
6.1 Linear Regression	20
6.2 Random Forest Regressor	20
6.3 Extreme Gradient Boosting (XGBoost)	21
6.4 Long Short-Term Memory (LSTM)	21
6.5 Model Performance Comparison	22
7. Development of Interactive Dashboard	23
Appendix	29

1. Project Overview

Electricity demand forecasting plays a vital role in ensuring the reliable operation and planning of modern power systems. Accurate forecasts enable electricity providers to balance generation with consumer demand, reduce operational costs, improve resource allocation, and support the integration of renewable energy sources. As electricity demand varies over time due to seasonal patterns, weather conditions, and consumer behaviour, machine learning techniques have become increasingly popular for producing accurate short-term forecasts from historical data.

This project develops and compares several machine learning models to forecast electricity demand using historical electricity generation and demand data collected by the Bangladesh Power Development Board (BPDB). The study follows a complete machine learning workflow consisting of data exploration, preprocessing, feature engineering, model development, performance evaluation, and visualization through an interactive Streamlit dashboard.

Four forecasting models were implemented and evaluated: Linear Regression, Random Forest Regressor, Extreme Gradient Boosting (XGBoost), and a Long Short-Term Memory (LSTM) neural network. Model performance was assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and the coefficient of determination (R^2). Finally, an interactive dashboard was developed to visualize historical electricity demand, compare model performance, and display forecasted values.

2. Dataset Overview

The dataset used in this project contains historical electricity system records covering the period from **April 2015 to June 2025**, comprising approximately **92,650 observations** collected at approximately hourly intervals. Each record represents the operational status of Bangladesh's electricity system at a particular point in time.

The dataset includes electricity demand together with multiple explanatory variables describing electricity generation and power sources. These include generation from gas, liquid fuel, coal, hydro, solar, and wind, as well as imported electricity from neighbouring countries such as India and Nepal. The dataset also contains information on load shedding and operational remarks.

The target variable selected for forecasting was **electricity demand (demand_mw)**, while the remaining numerical variables were used as predictor features. Exploratory Data Analysis (EDA) showed that the dataset contains several missing values, duplicate timestamps, and a number of extreme outliers. These issues were investigated before preprocessing to ensure

that appropriate cleaning decisions were made rather than removing information without justification.

Time-based features including **year, month, day, hour, day of week, and weekend indicators** were extracted from the datetime column. In addition, lag features (Demand Lag-1 and Demand Lag-24) together with a 24-hour rolling mean were created to capture temporal dependencies and improve forecasting performance.

Dataset Load

```
dataset = pd.read_csv('/content/drive/MyDrive/BEPRC Project/PCGB date power demand.csv')
dataset
```

	datetime	generation_mw	demand_mw	load_shedding	gas	liquid_fuel	coal	hydro	solar	wind	india_bheramara_hvdc	india_tripura	india_adani	nepal	remarks
0	2015-04-19 21:59:59	6323.0	6323	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
1	2015-04-19 21:00:00	6667.0	6667	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
2	2015-04-19 18:59:59	6897.0	6897	0	4415	1836	161	41	NaN	NaN	444	0	NaN	NaN	NaN
3	2015-04-19 18:30:00	6933.0	6933	0	4423	1862	159	45	NaN	NaN	444	0	NaN	NaN	Evening_Peak
4	2015-04-19 18:00:00	6874.0	6874	0	4319	1892	155	65	NaN	NaN	443	0	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
92645	2025-06-17 03:59:59	12698.0	12698	0	6030	834	3610	172	0.0	42.0	924	38	1010.0	38.0	NaN
92646	2025-06-17 03:00:00	13280.0	13280	0	6347	999	3698	172	0.0	40.0	924	38	1024.0	38.0	NaN
92647	2025-06-17 02:00:00	13719.0	13719	0	6340	1196	3881	172	0.0	27.0	924	36	1105.0	38.0	NaN
92648	2025-06-17 00:59:59	14093.0	14115	21	6323	1350	4019	172	0.0	28.0	924	38	1201.0	38.0	NaN
92649	2025-06-17 00:00:00	14628.0	14650	21	6302	1672	4186	172	0.0	41.0	924	42	1251.0	38.0	NaN

92650 rows x 15 columns

Figure: Dataset Loading

```
dataset.head(10)
```

	datetime	generation_mw	demand_mw	load_shedding	gas	liquid_fuel	coal	hydro	solar	wind	india_bheramara_hvdc	india_tripura	india_adani	nepal	remarks
0	2015-04-19 21:59:59	6323.0	6323	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
1	2015-04-19 21:00:00	6667.0	6667	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
2	2015-04-19 18:59:59	6897.0	6897	0	4415	1836	161	41	NaN	NaN	444	0	NaN	NaN	NaN
3	2015-04-19 18:30:00	6933.0	6933	0	4423	1862	159	45	NaN	NaN	444	0	NaN	NaN	Evening_Peak
4	2015-04-19 18:00:00	6874.0	6874	0	4319	1892	155	65	NaN	NaN	443	0	NaN	NaN	NaN
5	2015-04-19 17:29:59	6299.0	6299	0	4056	1602	156	42	NaN	NaN	443	0	NaN	NaN	NaN
6	2015-04-19 17:00:00	5725.0	5725	0	3892	1190	158	42	NaN	NaN	443	0	NaN	NaN	NaN
7	2015-04-19 15:59:59	5604.0	5604	0	3895	1065	160	40	NaN	NaN	444	0	NaN	NaN	NaN
8	2015-04-19 15:00:00	5651.0	5651	0	3942	1064	161	40	NaN	NaN	444	0	NaN	NaN	NaN
9	2015-04-19 14:00:00	5692.0	5692	0	3973	1074	162	40	NaN	NaN	443	0	NaN	NaN	NaN

```
#duplicate
dataset.duplicated().sum()

np.int64(160)
```

Figure: First 10 rows of dataset

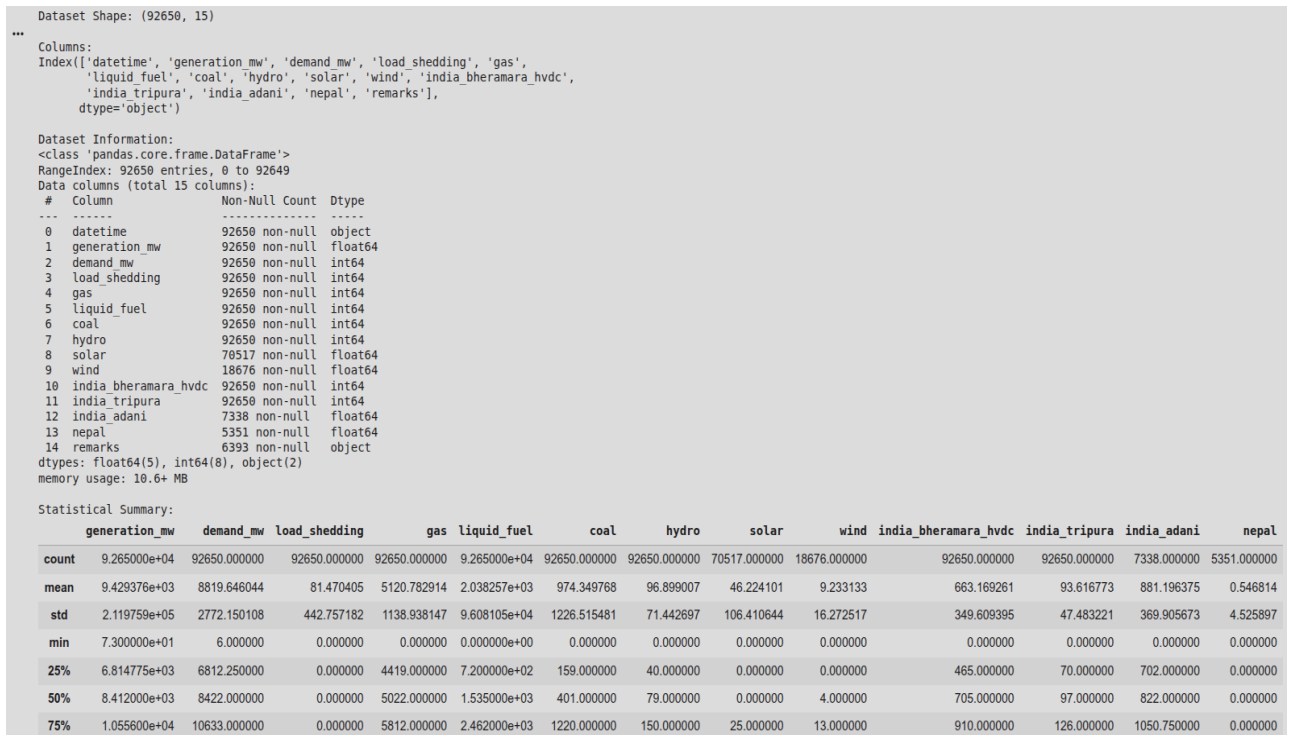


Figure: Dataset overview

3. Data Cleaning and Preprocessing

Before performing exploratory data analysis and model development, several preprocessing steps were carried out to ensure that the dataset was correctly structured for time-series forecasting.

3.1 Converting the Date Column

The `datetime` column was converted from string format to the Pandas `datetime` data type. This conversion enables efficient manipulation of date and time information and supports time-based operations such as indexing, filtering, and feature extraction.

```

...
      datetime
0  2015-04-19 21:59:59
1  2015-04-19 21:00:00
2  2015-04-19 18:59:59
3  2015-04-19 18:30:00
4  2015-04-19 18:00:00

dtype: datetime64[ns]

```

Figure: Converting 'datetime' column

3.2 Setting the Date Index

After conversion, the `datetime` column was set as the DataFrame index. Using a datetime index simplifies time-series analysis and allows chronological operations such as resampling, rolling statistics, lag feature creation, and visualization to be performed efficiently.

	generation_mw	demand_mw	load_shedding	gas	liquid_fuel	coal	hydro	solar	wind	india_bheramara_hvdc	india_tripura	india_adani	nepal	remarks
datetime														
2015-04-19 21:59:59	6323.0	6323	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
2015-04-19 21:00:00	6667.0	6667	0	0	0	0	0	NaN	NaN	0	0	NaN	NaN	NaN
2015-04-19 18:59:59	6897.0	6897	0	4415	1836	161	41	NaN	NaN	444	0	NaN	NaN	NaN
2015-04-19 18:30:00	6933.0	6933	0	4423	1862	159	45	NaN	NaN	444	0	NaN	NaN	Evening_Peak
2015-04-19 18:00:00	6874.0	6874	0	4319	1892	155	65	NaN	NaN	443	0	NaN	NaN	NaN

Figure: Setting Date Index

3.3 Sorting the Dataset

The dataset was sorted in ascending chronological order based on the datetime index. Maintaining the correct temporal sequence is essential for forecasting tasks because machine learning models should only learn from past observations when predicting future values.

```

dataset.sort_index(inplace=True)

print("Dataset sorted successfully.")

Dataset sorted successfully.

```

Figure: Sorting Dataset

3.4 Checking Chronological Order

A chronological integrity check was performed to verify that the observations were arranged correctly from the earliest to the latest timestamp. The dataset was confirmed to span **19**

April 2015 to 17 June 2025, ensuring that subsequent time-based train-test splitting preserved the natural order of the data and prevented information leakage.

```
print(dataset.index.is_monotonic_increasing)
```

True

Figure: Check chronological order

3.5 Checking Time Intervals

The time difference between consecutive observations was examined to understand the recording frequency of the dataset. The analysis showed that most records were collected at approximately **hourly intervals**, with smaller numbers of observations recorded at approximately **30-minute** and **2-hour** intervals. Minor differences of one second (e.g., 59 or 61 seconds) were observed due to timestamp recording precision rather than actual changes in sampling frequency. This confirmed that the dataset was suitable for short-term electricity demand forecasting.

```
dataset.index.to_series().diff().value_counts().head(10)
```

	count
0 days 01:00:01	29280
0 days 01:00:00	28739
0 days 00:59:59	25630
0 days 00:29:59	4164
0 days 00:30:00	3649
0 days 00:30:01	523
0 days 00:00:00	432
0 days 02:00:01	65
0 days 01:59:59	62
0 days 02:00:00	51

dtype: int64

Figure: Check time interval

3.6 Missing Value Analysis

The dataset was examined for missing values to determine whether any preprocessing was required before model development. Missing values were primarily found in the **solar**, **wind**, **india_adani**, **nepal**, and **remarks** columns. Investigation showed that these missing values were not caused by data corruption but rather reflected the fact that these power sources or imports were introduced into the electricity system at later dates. Consequently, missing numerical values were replaced with **0**, representing periods when the corresponding power source was not yet operational. The **remarks** column contained over 93% missing values and did not contribute to the forecasting task; therefore, it was removed from the dataset.

Based on the exploratory data analysis, the following preprocessing steps are applied before model development:

- Remove non-predictive columns.
- Replace structural missing values with zero.
- Verify that the dataset contains no remaining missing values.

```
# Remove non-predictive text column
dataset = dataset.drop(columns=["remarks"])

# Fill structural missing values with zero
fill_columns = ["solar", "wind", "india_adani", "nepal"]

dataset[fill_columns] = dataset[fill_columns].fillna(0)

# Verify missing values
missing = dataset.isnull().sum()

print("Remaining missing values:")
print(missing[missing > 0])

Remaining missing values:
Series([], dtype: int64)
```

Figure: Removing Missing values

3.7 Duplicate Value Investigation

Duplicate records were investigated to determine whether they represented true duplicated observations or legitimate measurements collected at different times. Two levels of analysis were performed: duplicate rows across all feature columns and duplicate datetime values. The investigation revealed that many apparent duplicates corresponded to repeated electricity measurements recorded at different timestamps rather than duplicated records. Only duplicate timestamps were considered problematic because each timestamp should uniquely represent a single observation in a time-series dataset. These duplicate timestamps were removed by retaining the first occurrence, preserving the chronological integrity of the dataset while preventing ambiguity during time-series analysis and forecasting.


```

duplicates = dataset.duplicated().sum()
print(f"Duplicate rows: {duplicates}")

Duplicate rows: 254

# Display some duplicate rows
duplicates = dataset[dataset.duplicated(keep=False)]
print(f"Number of duplicate rows: {duplicates.shape[0]}")
duplicates.head(10)

```

Number of duplicate rows: 470

	generation_mw	demand_mw	load_shedding	gas	liquid_fuel	coal	hydro	solar	wind	india_bheramara_hvdc	india_tripura	india_adani	nepal	remarks
datetime														
2015-04-30 00:00:00	6065.0	6065	0	4239	1187	156	32	NaN	NaN	451	0	NaN	NaN	NaN
2015-04-30 00:00:00	6889.0	6889	0	4698	1540	157	42	NaN	NaN	452	0	NaN	NaN	Evening_Peak
2015-04-30 17:29:59	6065.0	6065	0	4239	1187	156	32	NaN	NaN	451	0	NaN	NaN	NaN
2015-04-30 18:30:00	6889.0	6889	0	4698	1540	157	42	NaN	NaN	452	0	NaN	NaN	Evening_Peak
2015-05-06 00:00:00	7522.0	7522	0	4877	1984	151	46	NaN	NaN	464	0	NaN	NaN	NaN
2015-05-06 21:59:59	7522.0	7522	0	4877	1984	151	46	NaN	NaN	464	0	NaN	NaN	NaN
2015-05-09 00:00:00	5981.0	5981	0	4603	719	162	44	NaN	NaN	453	0	NaN	NaN	NaN
2015-05-09 12:00:00	5981.0	5981	0	4603	719	162	44	NaN	NaN	453	0	NaN	NaN	NaN
2015-05-17 00:00:00	7134.0	7134	0	4354	2098	150	79	NaN	NaN	453	0	NaN	NaN	NaN
2015-05-17 11:00:00	6275.0	6275	0	4175	1449	152	40	NaN	NaN	459	0	NaN	NaN	NaN

Figure: Duplicate value investigation

First value in case of duplicate timestamp

```
# Remove duplicate timestamps by keeping the first occurrence
dataset = dataset[~dataset.index.duplicated(keep='first')]

print("New dataset shape:", dataset.shape)
print("Remaining duplicate timestamps:", dataset.index.duplicated().sum())
```

```
New dataset shape: (92218, 14)
Remaining duplicate timestamps: 0
```

Check interval after processing duplicate values

```
dataset.index.to_series().diff().value_counts().head(10)
```

	count
datetime	
0 days 01:00:01	29280
0 days 01:00:00	28739
0 days 00:59:59	25630
0 days 00:29:59	4164
0 days 00:30:00	3649
0 days 00:30:01	523
0 days 02:00:01	65
0 days 01:59:59	62
0 days 02:00:00	51
0 days 03:00:00	11

```
dtype: int64
```

Figure: Check duplicate after removal

3.8 Removal of Non-Predictive Columns

The **remarks** column was removed from the dataset before model development. This column contained predominantly missing values and consisted of textual annotations describing operational events rather than quantitative measurements. Since it did not provide meaningful predictive information for forecasting electricity demand, retaining it would have unnecessarily increased the complexity of the dataset without improving model performance.

Based on the exploratory data analysis, the following preprocessing steps are applied before model development:

- Remove non-predictive columns.
- Replace structural missing values with zero.
- Verify that the dataset contains no remaining missing values.

```
# Remove non-predictive text column
dataset = dataset.drop(columns=["remarks"])

# Fill structural missing values with zero
fill_columns = ["solar", "wind", "india_adani", "nepal"]

dataset[fill_columns] = dataset[fill_columns].fillna(0)

# Verify missing values
missing = dataset.isnull().sum()

print("Remaining missing values:")
print(missing[missing > 0])

Remaining missing values:
Series([], dtype: int64)
```

Figure: Removing non-predictive column

3.9 Outlier Detection and Treatment

Exploratory Data Analysis revealed several extreme outliers in both electricity generation and demand, including unusually large values that were inconsistent with the normal operating range of the power system. These observations were investigated using time-series plots, histograms, boxplots, and scatter plots. Records identified as obvious data entry or recording anomalies were removed, while genuine high-demand observations were retained. This approach reduced the influence of erroneous values on model training while preserving realistic variations in electricity demand.

```
# Remove impossible generation value (>100,000 MW)

print("Rows before:", len(dataset))

dataset = dataset[dataset["generation_mw"] < 100000]

print("Rows after :", len(dataset))

Rows before: 92218
Rows after : 92217
```

Figure: Removing abnormal generational output (outlier values)

4. Exploratory Data Analysis (EDA)

4.1 Electricity Demand Over Time

A time-series plot of electricity demand was generated to examine long-term trends and seasonal behaviour. The analysis showed that electricity demand generally increased over the ten-year period, with recurring daily and seasonal fluctuations indicating clear temporal patterns. Several unusually large spikes were also observed, suggesting the presence of abnormal observations that required further investigation. This confirmed that electricity demand is highly time-dependent, supporting the use of time-based features and lag variables for forecasting.

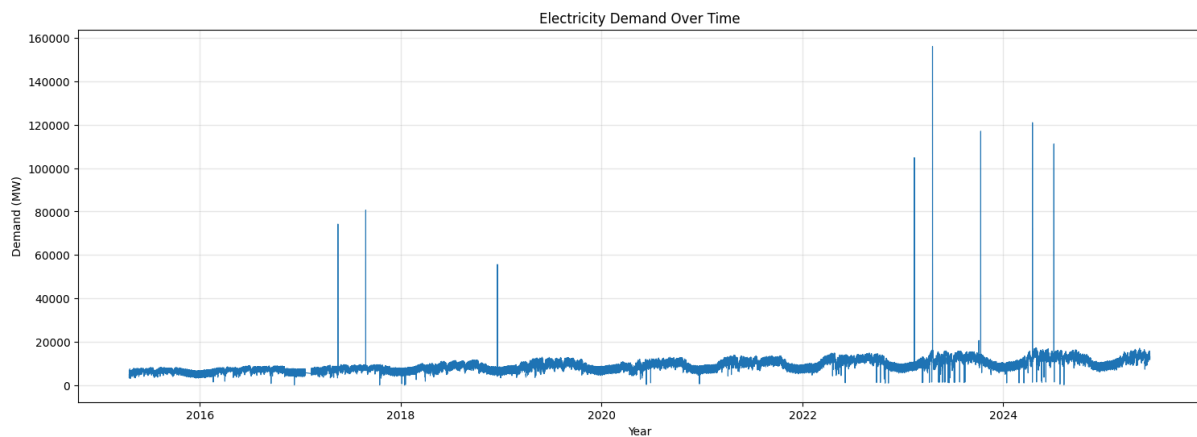


Figure: Electricity Demand over Time

4.2 Distribution of Electricity Demand

The distribution of electricity demand was examined using both a histogram and a boxplot. The histogram revealed that most demand values were concentrated within the normal operating range, while the distribution exhibited strong positive skewness due to a small number of extreme values. The boxplot further confirmed the presence of numerous high-value outliers extending far beyond the upper whisker. These visualisations highlighted the need to investigate whether the extreme observations represented genuine demand peaks or data recording errors before model training.

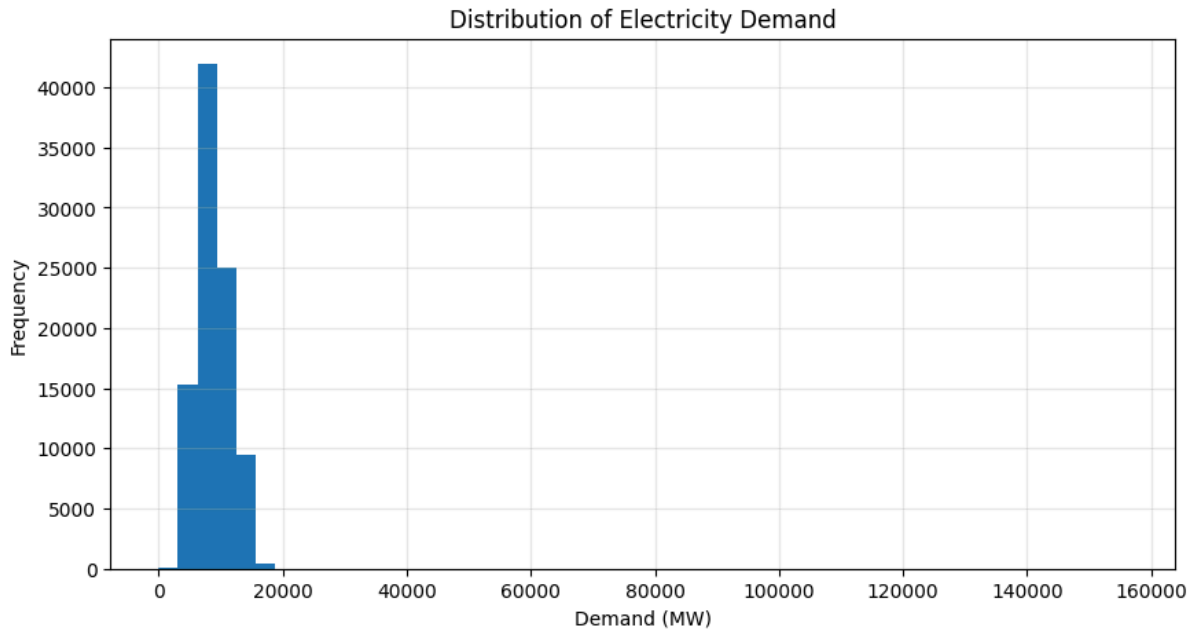


Figure: Histogram for Distribution of Electricity Demand

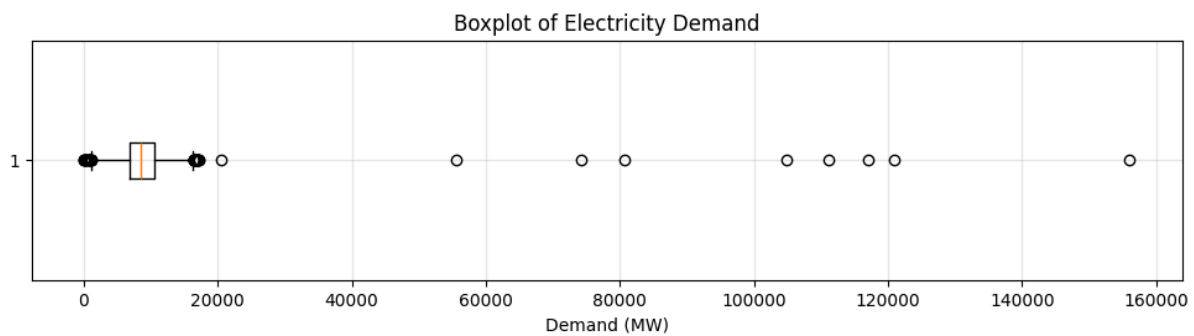


Figure: Boxplot for Distribution of Electricity Demand

4.3 Generation versus Electricity Demand

Scatter plots were used to analyse the relationship between electricity generation and electricity demand. The initial plot showed a distorted relationship caused by several extreme outliers. After removing erroneous observations, a clear positive linear relationship became evident, indicating that higher electricity generation generally corresponds to higher electricity demand. This strong relationship confirmed that generation is an important predictor for forecasting electricity demand and justified its inclusion as one of the primary input features.

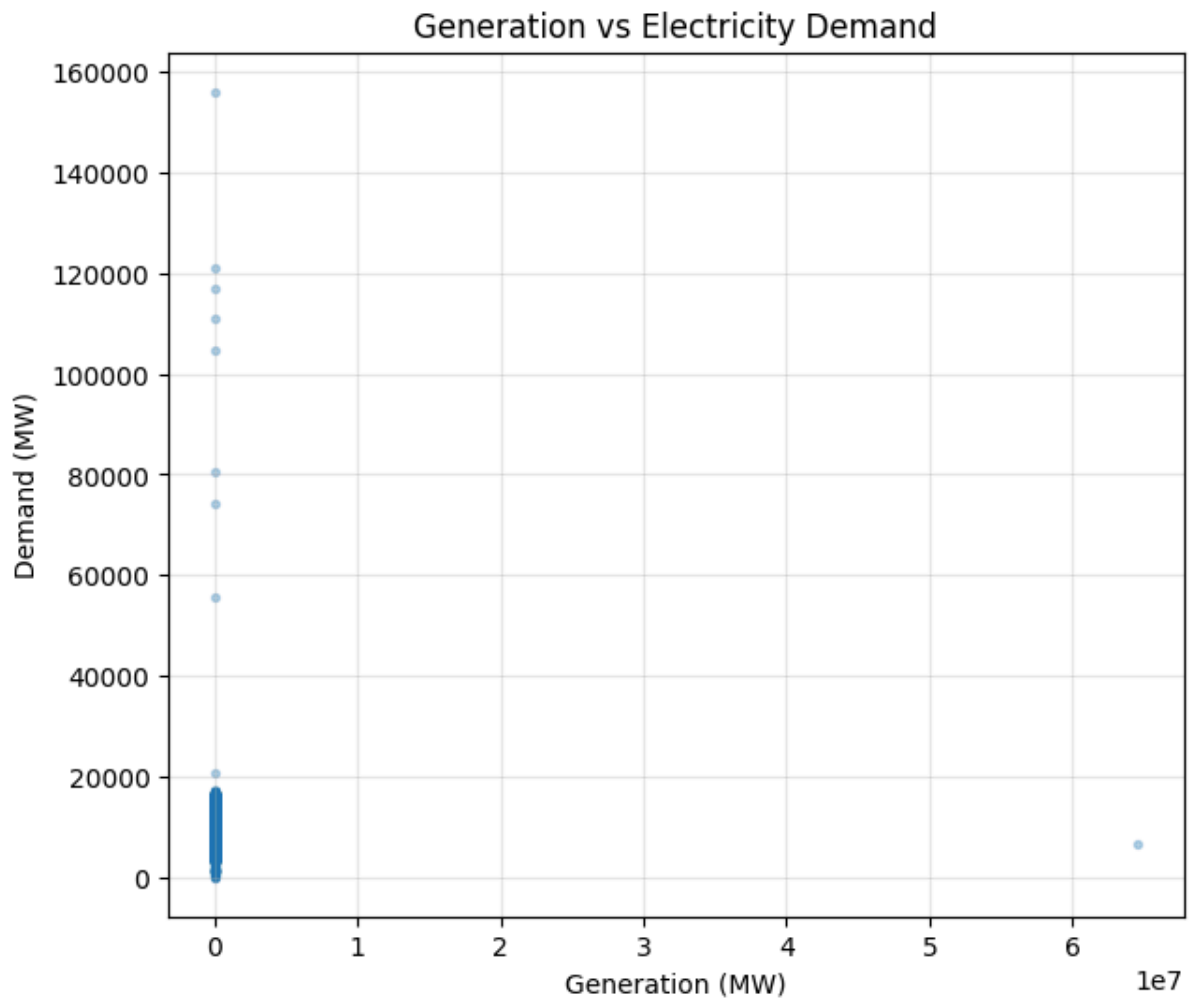


Figure: Generation vs. Demand with outlier

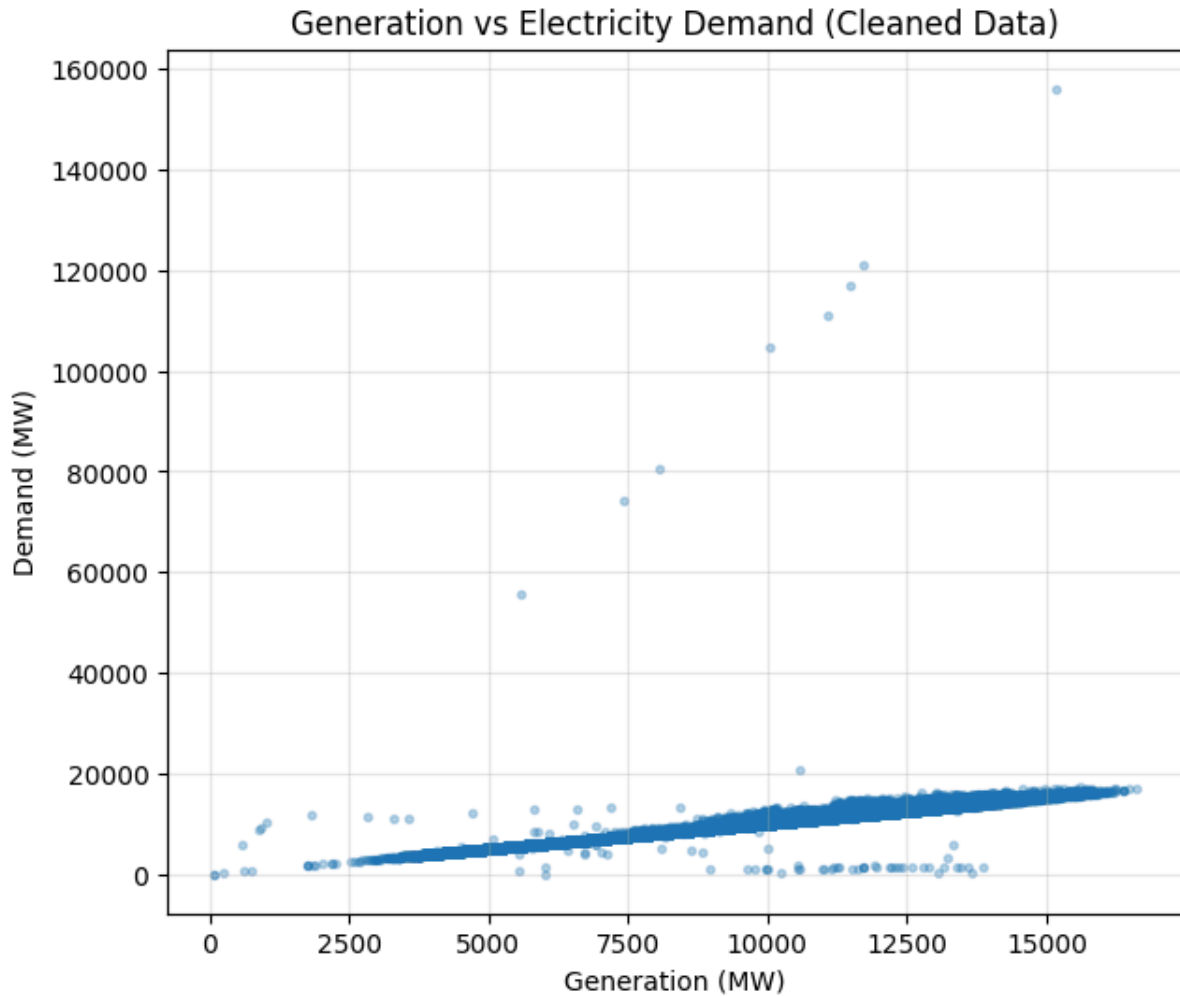


Figure: Generation vs. Demand without outlier

4.4 Correlation Analysis

A correlation matrix was generated to examine the relationships among the numerical variables. Electricity demand showed strong positive correlations with total electricity generation and several generation sources, while weaker relationships were observed for variables representing renewable energy and imported electricity. The analysis also indicated that no severe multicollinearity issues were present that would require feature removal. Overall, the correlation analysis supported the selection of the available numerical variables as predictors for the forecasting models.

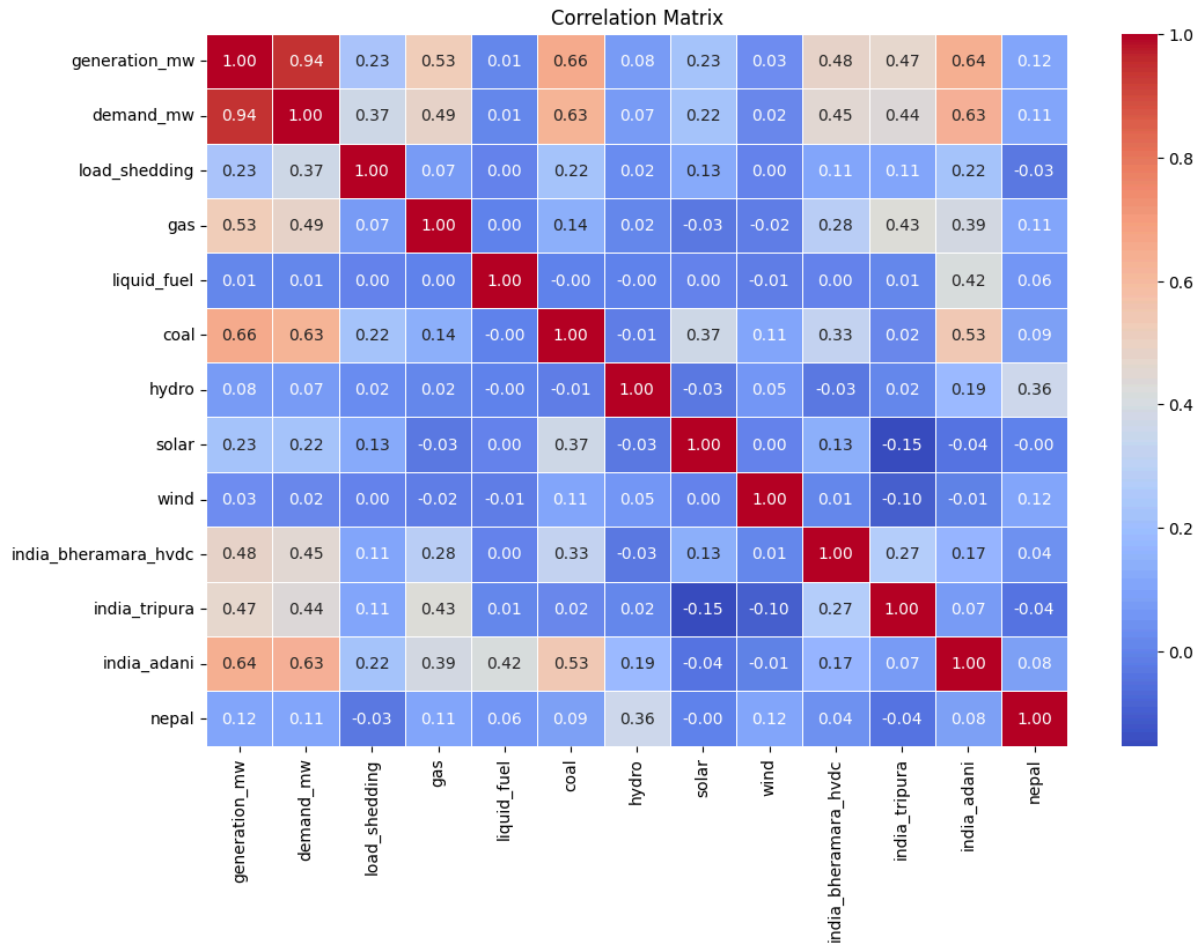


Figure: Confusion Matrix to establish correlation among features

5. Feature Engineering and Data Preparation

5.1 Time-Based Feature Extraction

Since electricity demand exhibits strong temporal patterns, additional time-related features were extracted from the datetime index. These included **year**, **month**, **day**, **hour**, **day of the week**, and a **weekend indicator**. These features allow the models to learn recurring daily, weekly, and seasonal variations in electricity demand.


```
# Create time-based features from the datetime index

dataset["year"] = dataset.index.year
dataset["month"] = dataset.index.month
dataset["day"] = dataset.index.day
dataset["hour"] = dataset.index.hour
dataset["day_of_week"] = dataset.index.dayofweek

# Weekend indicator (Saturday = 5, Sunday = 6)
dataset["is_weekend"] = dataset["day_of_week"].isin([5, 6]).astype(int)

# Preview the new features
dataset[
    ["year", "month", "day", "hour", "day_of_week", "is_weekend"]
].head()
```

	year	month	day	hour	day_of_week	is_weekend
datetime						
2015-04-19 00:00:00	2015	4	19	0	6	1
2015-04-19 00:59:59	2015	4	19	0	6	1
2015-04-19 02:00:00	2015	4	19	2	6	1
2015-04-19 03:00:00	2015	4	19	3	6	1
2015-04-19 03:59:59	2015	4	19	3	6	1

Figure: Time based feature extraction

5.2 Lag and Rolling Features

To capture temporal dependencies, lag features were created using the electricity demand values from the **previous hour (Lag-1)** and the **previous 24 hours (Lag-24)**. In addition, a **24-hour rolling mean** was calculated to provide a smoothed representation of recent demand trends. These features help both machine learning and deep learning models utilize historical demand information when forecasting future electricity demand.

```

# Create lag features

dataset["demand_lag_1"] = dataset["demand_mw"].shift(1)
dataset["demand_lag_24"] = dataset["demand_mw"].shift(24)

# Create rolling mean feature
dataset["rolling_mean_24"] = (
    dataset["demand_mw"]
    .rolling(window=24)
    .mean()
)

# Check missing values introduced by lagging
print(dataset[
    ["demand_lag_1", "demand_lag_24", "rolling_mean_24"]
].isnull().sum())

```

demand_lag_1	1
demand_lag_24	24
rolling_mean_24	23
dtype:	int64

Figure: Creating lag features

5.3 Removal of Missing Rows

The creation of lag and rolling features introduced missing values at the beginning of the dataset because previous observations were unavailable for the first few records. Since these missing values were a natural consequence of feature engineering and represented only a small portion of the dataset, the affected rows were removed using `dropna()` before model training.

5.4 Data Preparation for Modelling

The target variable for this study was **electricity demand (demand_mw)**, while the remaining variables were used as predictor features. The dataset was divided into input features (**X**) and the target variable (**y**). An **80:20 time-based train-test split** was then applied without shuffling to preserve the chronological order of the observations and prevent data leakage, ensuring that the models were trained on historical data and evaluated on unseen future data.

Data Modelling Preparation

```
# Features and target
X = dataset.drop(columns=["demand_mw"])
y = dataset["demand_mw"]

# 80% train, 20% test (chronological split)

split_index = int(len(dataset) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]

y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

Training samples: 73754
Testing samples : 18439
```

Figure: Data Modelling Preparation for all classical models

5.5 Data Scaling for LSTM

Unlike the classical machine learning models, the LSTM network requires input data to be on a similar numerical scale to facilitate efficient training and convergence. Therefore, the input features and target variable were normalized using **Min-Max Scaling** before training the LSTM model. After prediction, the forecasted values were transformed back to their original scale to allow meaningful evaluation and comparison with the actual electricity demand values.

```

# Scale features
feature_scaler = MinMaxScaler()
X_scaled = feature_scaler.fit_transform(X)

# Scale target
target_scaler = MinMaxScaler()
y_scaled = target_scaler.fit_transform(y.values.reshape(-1, 1))

sequence_length = 24

X_lstm = []
y_lstm = []

for i in range(sequence_length, len(X_scaled)):
    X_lstm.append(X_scaled[i-sequence_length:i])
    y_lstm.append(y_scaled[i])

X_lstm = np.array(X_lstm)
y_lstm = np.array(y_lstm)

print("X shape:", X_lstm.shape)
print("y shape:", y_lstm.shape)

X shape: (92169, 24, 21)
y shape: (92169, 1)

split_index = int(len(X_lstm) * 0.8)

X_train_lstm = X_lstm[:split_index]
X_test_lstm = X_lstm[split_index:]

y_train_lstm = y_lstm[:split_index]
y_test_lstm = y_lstm[split_index:]

print("Training samples:", len(X_train_lstm))
print("Testing samples :", len(X_test_lstm))

Training samples: 73735
Testing samples : 18434

```

Figure: Data scaling and preparation for Deep Learning model (LSTM)

6. Model Development

To evaluate the effectiveness of different forecasting approaches, four predictive models were developed and compared: **Linear Regression**, **Random Forest Regressor**, **Extreme Gradient Boosting (XGBoost)**, and a **Long Short-Term Memory (LSTM)** neural network.

The first three represent classical machine learning techniques, while the LSTM represents a deep learning approach designed specifically for sequential time-series forecasting.

All models were trained using the same preprocessed dataset and evaluated using an 80:20 time-based train-test split without shuffling. Model performance was assessed using **Mean Absolute Error (MAE)**, **Root Mean Squared Error (RMSE)**, **Mean Absolute Percentage Error (MAPE)**, and the **Coefficient of Determination (R^2)**.

6.1 Linear Regression

Linear Regression was implemented as a baseline forecasting model to establish a benchmark for comparison. Despite its simplicity, the model produced competitive results with an **MAE of 363.01 MW**, **RMSE of 1545.74 MW**, **MAPE of 5.16%**, and an **R^2 score of 0.6715**. The results indicate that electricity demand exhibits a strong linear relationship with the selected predictor variables, making Linear Regression an effective baseline model.

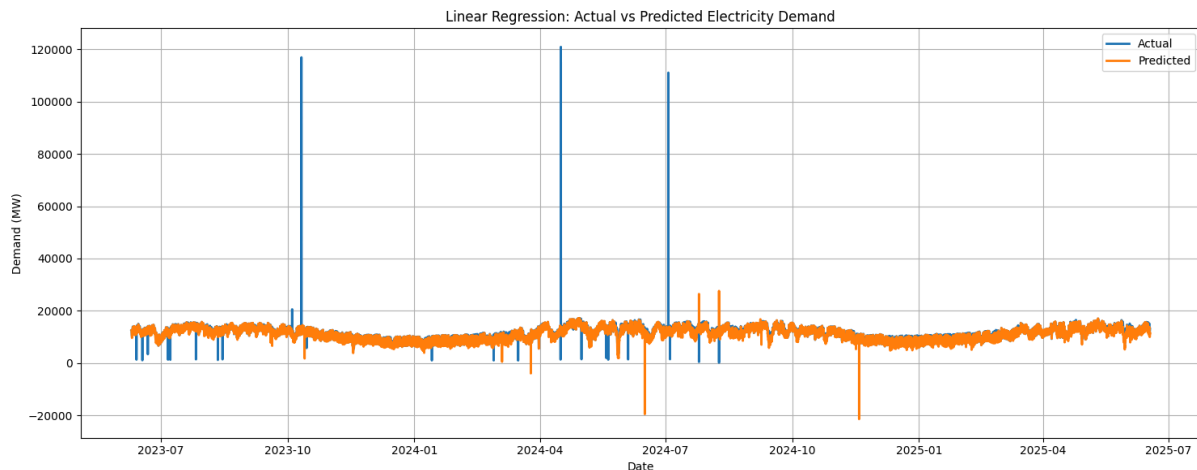


Figure: Actual vs. Predicted (Electricity Demand) for Linear Regression

6.2 Random Forest Regressor

Random Forest was selected because of its ability to model complex, non-linear relationships while remaining robust to noise in the data. Among all evaluated models, it achieved the **lowest MAE (318.73 MW)** and the **lowest MAPE (3.41%)**, indicating the highest average prediction accuracy. However, its **RMSE of 1599.18 MW** and **R^2 score of 0.6483** were slightly lower than those of Linear Regression and LSTM, suggesting that larger prediction errors occurred more frequently.

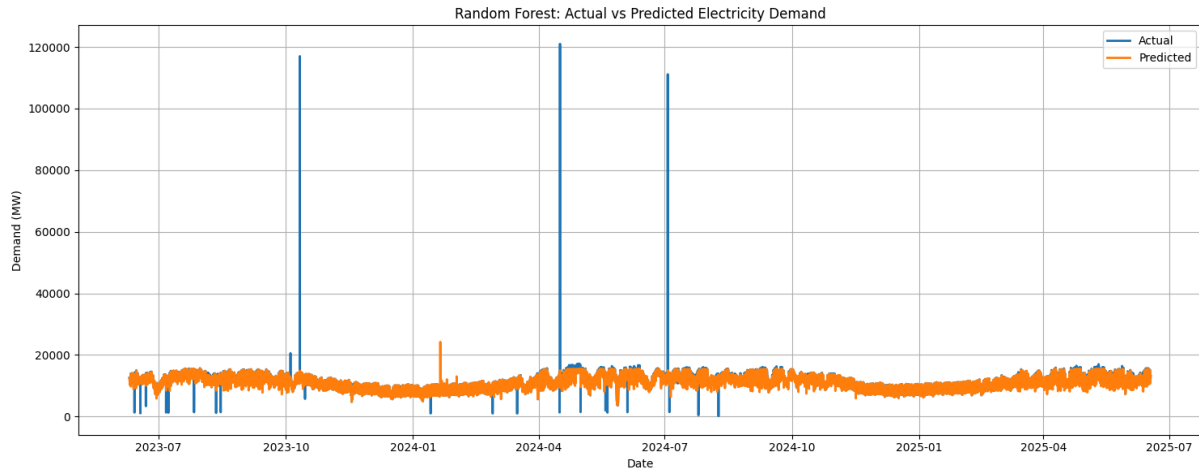


Figure: Actual vs. Predicted (Electricity Demand) for Random Forest Regressor

6.3 Extreme Gradient Boosting (XGBoost)

XGBoost was included due to its strong reputation for regression tasks and ensemble learning. However, it produced the weakest performance in this study, with an **MAE of 562.41 MW**, **RMSE of 1704.87 MW**, **MAPE of 5.89%**, and an **R^2 score of 0.6003**. Although the model successfully captured the overall trend of electricity demand, it did not outperform the other forecasting models on this dataset.

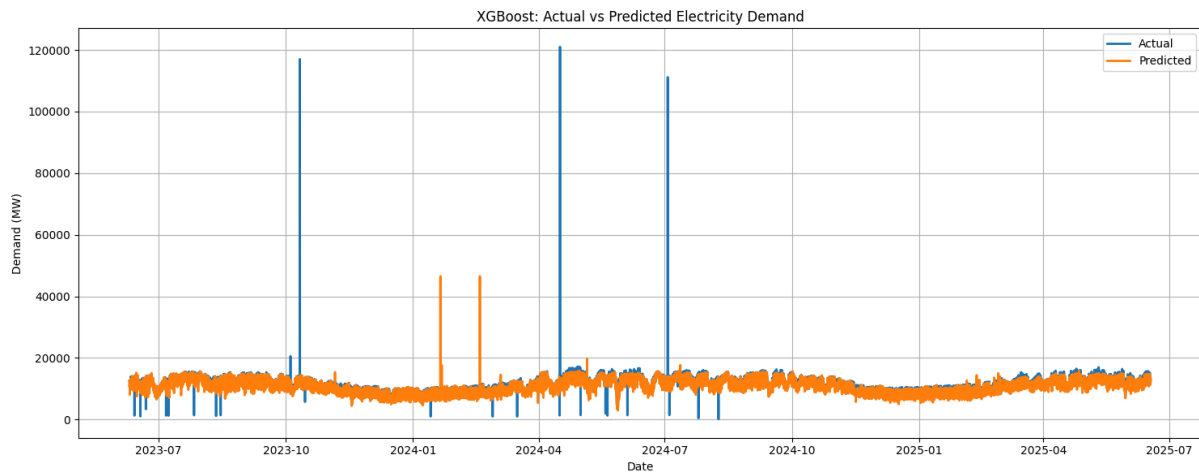


Figure: Actual vs. Predicted (Electricity Demand) for XGboost

6.4 Long Short-Term Memory (LSTM)

An LSTM neural network was developed to capture temporal dependencies within the electricity demand time series. Historical demand sequences were used as inputs, and Min-Max normalization was applied before training. Early stopping was incorporated during training to reduce overfitting and improve generalization. The LSTM achieved the **lowest RMSE (1514.40 MW)** and the **highest R^2 score (0.6847)**, demonstrating the best overall forecasting performance. While its **MAE (484.95 MW)** and **MAPE (5.58%)** were higher

than those of Random Forest, the model provided the strongest overall fit to the data and produced the most accurate predictions when considering overall variance.

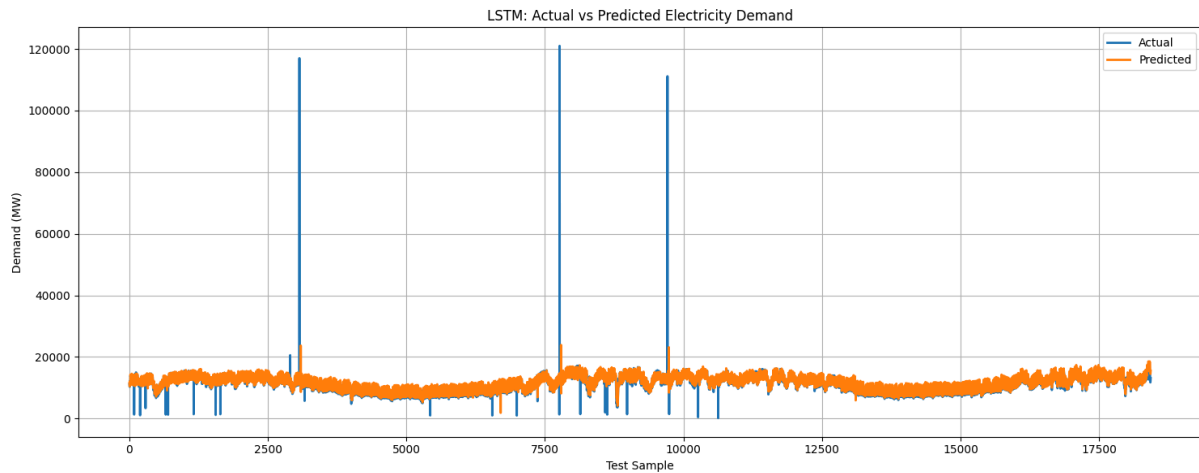


Figure: Actual vs. Predicted (Electricity Demand) for LSTM

6.5 Model Performance Comparison

Table-1, below summarises the performance of the four forecasting models.

Model	MAE (MW)	RMSE (MW)	MAPE (%)	R ²
Linear Regression	363.01	1545.74	5.16	0.6715
Random Forest	318.73	1599.18	3.41	0.6483
XGBoost	562.41	1704.87	5.89	0.6003
LSTM	484.95	1514.40	5.58	0.6847

Table-1: Model comparison based on different metrics

Overall, the **LSTM model** achieved the best overall forecasting performance, recording the lowest RMSE and the highest R² score, indicating that it explained the greatest proportion of variation in electricity demand while producing the smallest large-scale prediction errors. In contrast, the **Random Forest model** achieved the lowest MAE and MAPE, demonstrating superior average prediction accuracy across the test set. Linear Regression also performed remarkably well despite its simplicity, outperforming XGBoost on every evaluation metric. These findings suggest that while deep learning provides the most accurate overall forecasts, simpler machine learning models remain highly effective for electricity demand prediction and can offer competitive performance with significantly lower computational complexity.

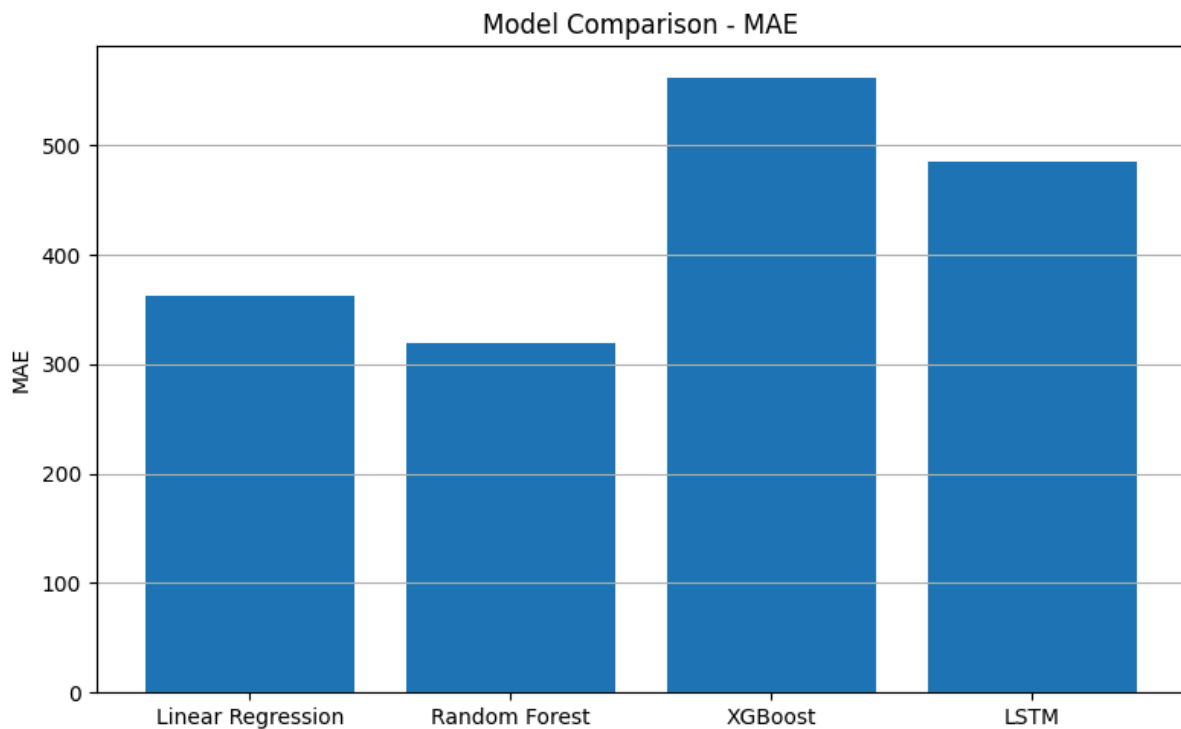


Figure: Comparison bar chart for all the models (MAE Score)

7. Development of Interactive Dashboard

To complement the forecasting models, a simple interactive web dashboard was developed using **Streamlit**. The dashboard provides an accessible interface for visualising the dataset, comparing model performance, and exploring forecasting results without requiring users to interact with the Python notebook.

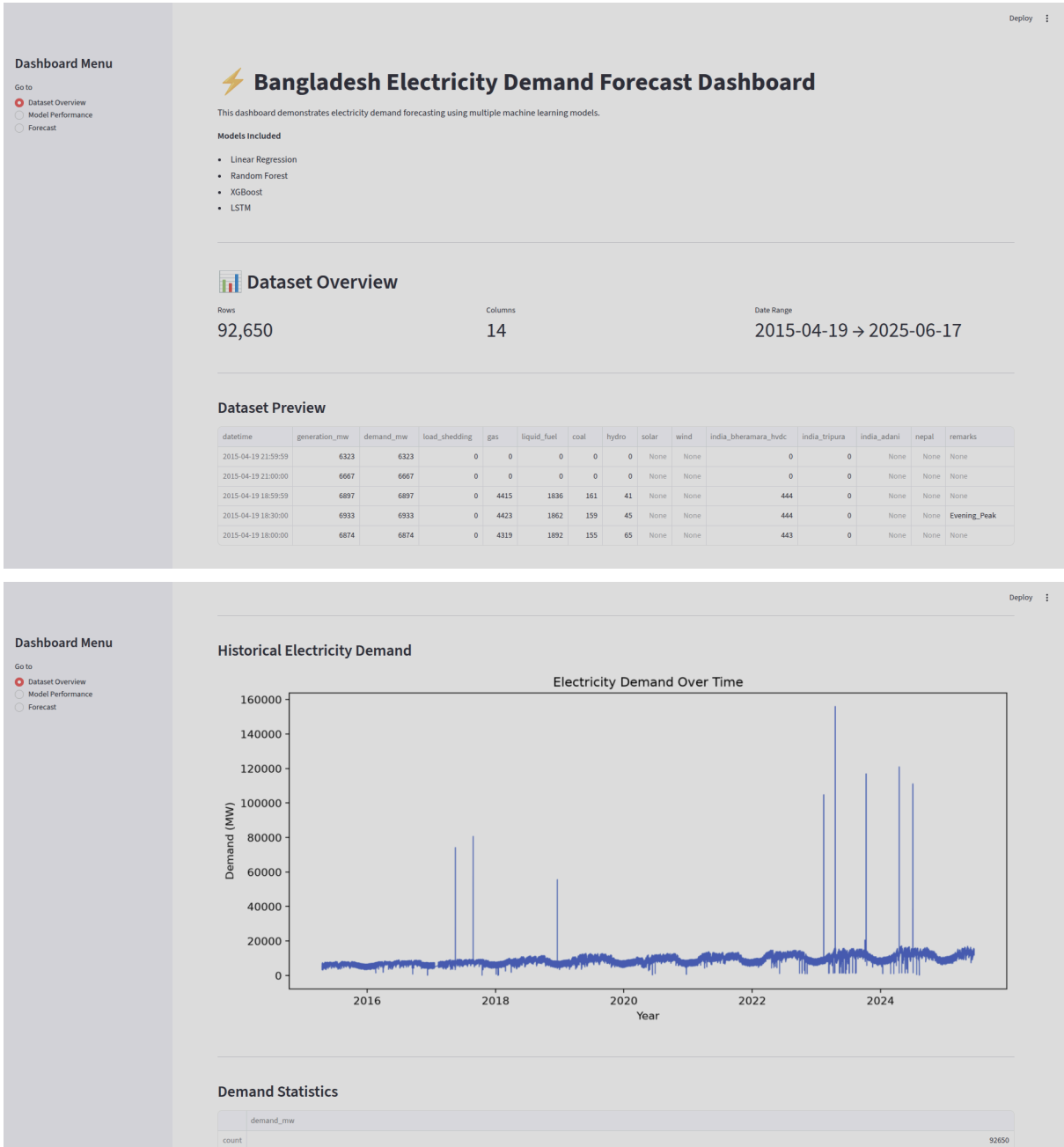
The dashboard was organised into three main sections. The **Dataset Overview** page presents a time-series visualization of the historical electricity demand along with a summary of the dataset, allowing users to explore the underlying data. The **Model Performance** page displays the evaluation metrics (MAE, RMSE, MAPE, and R^2) for all developed models in both tabular and graphical formats, enabling straightforward comparison of forecasting accuracy. The best-performing model is also highlighted based on the evaluation results.

The **Forecast** page allows users to select one of the four trained models (Linear Regression, Random Forest, XGBoost, or LSTM) using an interactive drop-down menu. Once a model is selected, the dashboard displays key summary statistics, including the latest actual demand, latest predicted demand, and the model's RMSE. An interactive line chart overlays the actual and predicted electricity demand values, enabling users to visually assess the forecasting performance over the selected portion of the test dataset.

Overall, the dashboard satisfies the project requirements by providing a historical time-series visualization, forecast comparison, summary statistics, and interactive controls through a

lightweight and user-friendly web application.

Screenshots:



[illegible]

Dashboard Menu

Go to

Dataset Overview

Model Performance

Forecast

	Model	MAE	RMSE	MAPE	R2	
0	Linear Regression	363.01	363.01	1545.74	5.16	0.6715
1	Random Forest	318.73	318.73	1599.18	3.41	0.6483
2	XGBoost	562.41	562.41	1704.87	5.89	0.6003
3	LSTM	484.95	484.95	1514.40	5.58	0.6847

Best Performing Model: LSTM

Best RMSE

1514.40

Best MAE

318.73

Model Comparison

Select Evaluation Metric

MAE

MAE Comparison Across Models

500

562.41

484.95

Deploy

Dashboard Menu

Go to

☐ Dataset Overview

☒ Model Performance

☐ Forecast

	Model	MAE	RMSE	MAPE	R2	
0	Linear Regression	363.01	318.73	1545.74	5.16	0.0715
1	Random Forest		318.73	1599.18	3.41	0.6483
2	XGBoost		562.41	1704.87	5.89	0.6003
3	LSTM		484.95	1514.40	5.58	0.6847

Best Performing Model: LSTM

Best RMSE

1514.40

Best MAE

318.73

Model Comparison

Select Evaluation Metric

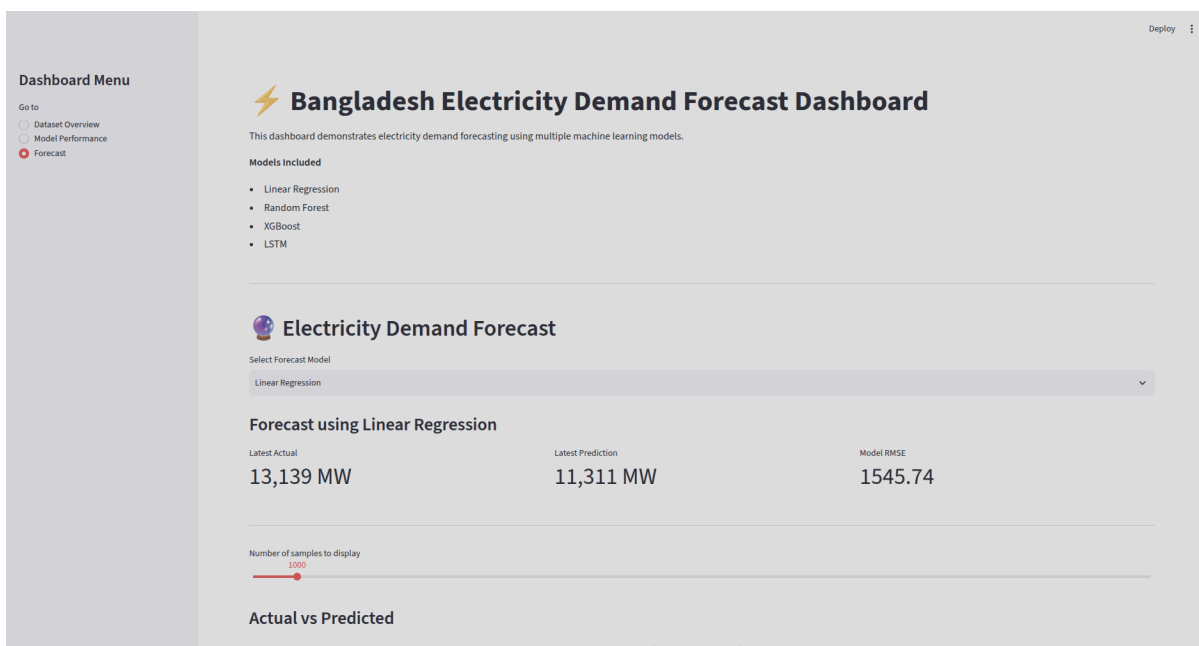
MAE

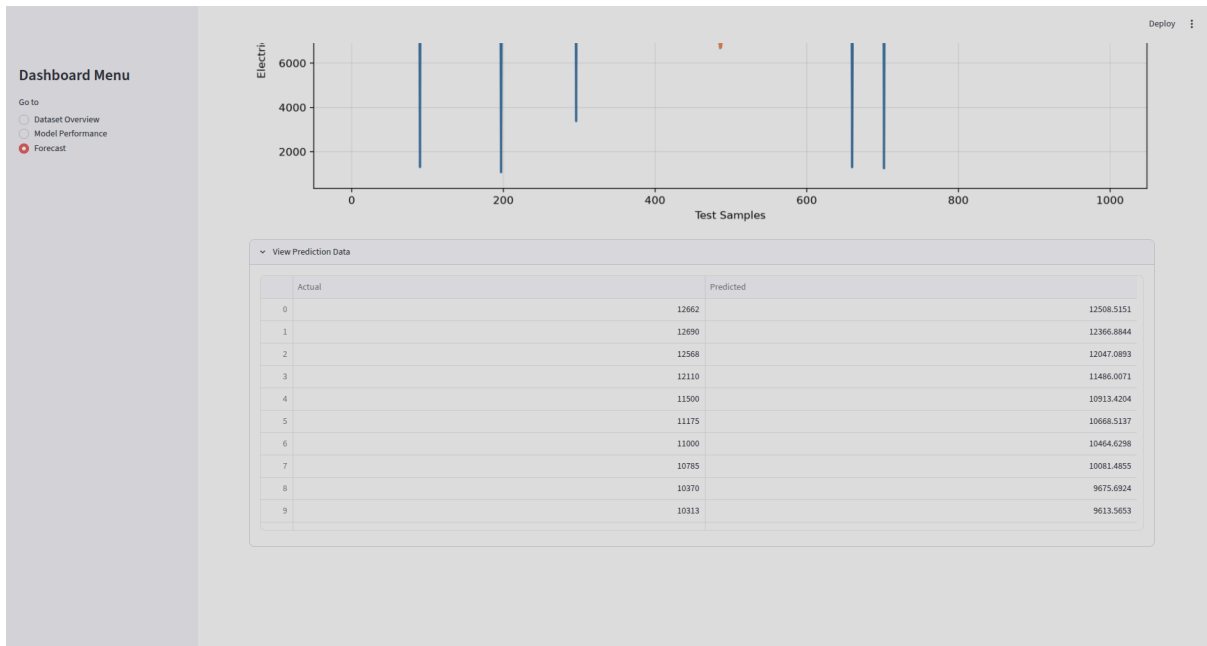
MAE Comparison Across Models

500

562.41

484.95





Appendix

Code for Web dashboard development:

```
import matplotlib.pyplot as plt
import pandas as pd
import streamlit as st

# Load dataset
dataset = pd.read_csv("PGCB_date_power_demand.csv")

# Load model evaluation results
results = pd.read_csv("results.csv")

# Prediction files
lr_pred = pd.read_csv("lr_predictions.csv")
rf_pred = pd.read_csv("rf_predictions.csv")
xgb_pred = pd.read_csv("xgb_predictions.csv")
lstm_pred = pd.read_csv("lstm_predictions.csv")

# Convert datetime
dataset["datetime"] = pd.to_datetime(dataset["datetime"])

# Set datetime as index
dataset.set_index("datetime", inplace=True)

# Page configuration
st.set_page_config(
    page_title="Bangladesh Electricity Demand Forecast Dashboard",
    page_icon="⚡",
    layout="wide",
)

# Title
st.title("⚡ Bangladesh Electricity Demand Forecast Dashboard")

st.markdown("""
This dashboard demonstrates electricity demand forecasting using multiple machine
learning models.

**Models Included**
```

- Linear Regression
- Random Forest
- XGBoost
- LSTM

""")

st.divider()

Sidebar

st.sidebar.title("Dashboard Menu")

page = st.sidebar.radio("Go to", ["Dataset Overview", "Model Performance", "Forecast"])

if page == "Dataset Overview":

st.header("📊 Dataset Overview")

Summary metrics

col1, col2, col3 = st.columns(3)

col1.metric("Rows", f"{dataset.shape[0]:,}")

col2.metric("Columns", dataset.shape[1])

col3.metric(
 "Date Range", f"{dataset.index.min().date()} → {dataset.index.max().date()}"
)

st.divider()

st.subheader("Dataset Preview")

st.dataframe(dataset.head())

st.divider()

st.subheader("Historical Electricity Demand")

fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(dataset.index, dataset["demand_mw"], color="royalblue", linewidth=0.8)

ax.set_xlabel("Year")

ax.set_ylabel("Demand (MW)")

ax.set_title("Electricity Demand Over Time")

st.pyplot(fig)

```

st.divider()

st.subheader("Demand Statistics")

st.dataframe(dataset["demand_mw"].describe())

elif page == "Model Performance":
    st.header("📈 Model Performance")

    # -----
    # Evaluation Table
    # -----
    st.subheader("Evaluation Metrics")

    st.dataframe(
        results.style.format(
            {"MAE": "{:.2f}", "RMSE": "{:.2f}", "MAPE": "{:.2f}", "R2": "{:.4f}"},
        ),
        use_container_width=True,
    )

    st.divider()

    # -----
    # Best Model
    # -----
    best_model = results.loc[results["RMSE"].idxmin(), "Model"]

    st.success(f"🏆 Best Performing Model: {best_model}")

    col1, col2 = st.columns(2)

    col1.metric("Best RMSE", f"{results['RMSE'].min():.2f}")

    col2.metric("Best MAE", f"{results['MAE'].min():.2f}")

    st.divider()

    # -----
    # Interactive Comparison Chart
    # -----
    st.subheader("📊 Model Comparison")

```



```

metric = st.selectbox(
    "Select Evaluation Metric", ["MAE", "RMSE", "MAPE", "R2"],
    key="metric_selector"
)

fig, ax = plt.subplots(figsize=(8, 5))

bars = ax.bar(results["Model"], results[metric])

if metric == "R2":
    best_idx = results[metric].idxmax()
else:
    best_idx = results[metric].idxmin()

bars[best_idx].set_color("green")

ax.set_ylabel(metric)
ax.set_title(f"{metric} Comparison Across Models")

plt.xticks(rotation=15)

for bar in bars:
    height = bar.get_height()

    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.2f}",
        ha="center",
        va="bottom",
        fontsize=9,
    )

st.pyplot(fig)

elif page == "Forecast":
    st.header("🌐 Electricity Demand Forecast")

    # -----
    # Select Forecast Model
    # -----
    model_choice = st.selectbox(

```

```

    "Select Forecast Model",
    ["Linear Regression", "Random Forest", "XGBoost", "LSTM"],
    key="forecast_model",
)

# Load selected model predictions
if model_choice == "Linear Regression":
    pred_df = lr_pred
    rmse = results.loc[results["Model"] == "Linear Regression", "RMSE"].values[0]

elif model_choice == "Random Forest":
    pred_df = rf_pred
    rmse = results.loc[results["Model"] == "Random Forest", "RMSE"].values[0]

elif model_choice == "XGBoost":
    pred_df = xgb_pred
    rmse = results.loc[results["Model"] == "XGBoost", "RMSE"].values[0]

else:
    pred_df = lstm_pred
    rmse = results.loc[results["Model"] == "LSTM", "RMSE"].values[0]

# -----
# Summary Cards
# -----
st.subheader(f"Forecast using {model_choice}")

col1, col2, col3 = st.columns(3)

col1.metric("Latest Actual", f"{pred_df['Actual'].iloc[-1]:.0f} MW")

col2.metric("Latest Prediction", f"{pred_df['Predicted'].iloc[-1]:.0f} MW")

col3.metric("Model RMSE", f"{rmse:.2f}")

st.divider()

# -----
# Interactive Sample Selector
# -----
num_samples = st.slider(
    "Number of samples to display",
    min_value=100,

```

```

    max_value=len(pred_df),
    value=1000,
    step=100,
)

plot_df = pred_df.head(num_samples)

# -----
# Actual vs Predicted Plot
# -----
st.subheader("Actual vs Predicted")

fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(plot_df["Actual"], label="Actual", linewidth=2)

ax.plot(plot_df["Predicted"], label="Predicted", linewidth=2)

ax.set_title(f"{model_choice} Forecast")
ax.set_xlabel("Test Samples")
ax.set_ylabel("Electricity Demand (MW)")

ax.legend()

ax.grid(alpha=0.3)

st.pyplot(fig)

# -----
# Prediction Table
# -----
with st.expander("View Prediction Data"):
    st.dataframe(plot_df, use_container_width=True)

```